

BI3006 - Data Warehouse y Dashboard de Ventas

Compilacion reproducible del proyecto

Este documento compila el contenido del README principal, la documentacion del dashboard y la bitacora de carga para dejar una guia unica y reproducible del trabajo.

Proyecto	BI3006 - Data Warehouse y Dashboard de Ventas
Plataforma	Windows / Linux / macOS
Objetivo	Construir, validar y visualizar un Data Warehouse en SQLite con Streamlit
Fuentes compiladas	README.md, documentacion/dashboard_documentacion.md, documentacion/bitacora_carga.txt

1. Descripcion general

El proyecto implementa un Data Warehouse en SQLite, ejecuta validaciones de consistencia y muestra indicadores de negocio en un dashboard interactivo desarrollado con Streamlit.

El flujo completo incluye carga inicial, carga incremental, pruebas de consistencia, validacion de KPIs y visualizacion final en navegador.

2. Estructura del proyecto

```
VideoJuegos/  
??? dashboard/  
?   ??? dashboard_dw.py  
?   ??? requirements.txt  
?   ??? run_dashboard.bat  
??? documentacion/  
?   ??? bitacora_carga.txt  
?   ??? dashboard_documentacion.md  
?   ??? logs/  
??? scripts/  
?   ??? carga_dw.py  
?   ??? kpis_validacion.py  
?   ??? pruebas_consistencia.py  
??? DimJuego.csv  
??? DimPlataforma.csv  
??? DimRatingCritico.csv  
??? DimTiempo.csv  
??? FactVentas.csv  
??? data_warehouse.db
```

3. Requisitos

Python	3.10+
pip	20.0+

SQLite	Incluido en Python
Sistema operativo	Windows, Linux o macOS

La base de datos esperada es **data_warehouse.db** en la raíz del proyecto. Si no existe, debe generarse con `scripts/carga_dw.py`.

4. Instalacion

Desde la raíz del proyecto:

```
pip install -r dashboard/requirements.txt
```

No se debe agregar `sqlite3` al archivo de dependencias porque forma parte de la librería estándar de Python.

5. Ejecucion paso a paso

Paso 1: Cargar el Data Warehouse

```
python scripts/carga_dw.py
```

- Carga la dimension de juegos desde `DimJuego.csv`
- Genera `DimTiempo` (2000-2024)
- Genera `DimCliente` (100 clientes)
- Genera `Hecho_Ventas` (10000 registros)
- Simula carga incremental de ventas
- Verifica integridad referencial básica

Paso 2: Ejecutar pruebas de consistencia

```
python scripts/pruebas_consistencia.py
```

- Valida integridad referencial entre hechos y dimensiones
- Valida precios y cantidades no negativas
- Detecta duplicados en claves de negocio

Paso 3: Calcular y validar KPIs

```
python scripts/kpis_validacion.py
```

- Ventas por mes
- Clientes activos por region
- Top productos vendidos
- Ticket promedio
- Conteos generales de tablas

Paso 4: Iniciar dashboard

```
streamlit run dashboard/dashboard_dw.py
```

- Abre el dashboard en `http://localhost:8501`
- El arranque deja evidencia en `documentacion/logs/dashboard.log`

6. Bitacora y logs

La evidencia principal se encuentra en documentacion/bitacora_carga.txt. Ese archivo resume la corrida con marcas de tiempo, duracion, verificaciones y referencia a los logs completos.

carga_dw	documentacion/logs/carga_dw.log
pruebas_consistencia	documentacion/logs/pruebas_consistencia.log
kpis_validacion	documentacion/logs/kpis_validacion.log
dashboard	documentacion/logs/dashboard.log

7. Dashboard y KPIs

El dashboard se construye con Streamlit y Plotly. Expone filtros por fecha, genero, publisher y region, y recalcula los KPI visibles en tiempo real.

KPIs principales:

- Ventas Totales: SUM(Total)
- Transacciones: COUNT(*)
- Clientes Unicos: COUNT(DISTINCT ID_Cliente)
- Juegos Vendidos: COUNT(DISTINCT ID_Juego)
- Ticket Promedio: AVG(Total)

8. Troubleshooting

- No se encuentra DimJuego.csv: Verificar que el archivo exista en la raiz del proyecto con ese nombre exacto.
- No se encuentra data_warehouse.db: Ejecutar python scripts/carga_dw.py para generar la base.
- Puerto 8501 ocupado: Usar streamlit run dashboard/dashboard_dw.py --server.port 8502
- Faltan dependencias de Streamlit: Ejecutar pip install -r dashboard/requirements.txt

9. Documentacion tecnica ampliada

La documentacion del dashboard agrega detalles de arquitectura, stack tecnologico, optimizaciones y mantenimiento. La compilacion completa considera esta informacion para facilitar la reproduccion del trabajo.

Arquitectura:

Dashboard (Streamlit)

?

Conexion SQLite

?

Data Warehouse (data_warehouse.db)

??? DimJuego

??? DimTiempo

??? DimCliente

??? DimPlataforma

??? DimRatingCritico

??? Hecho_Ventas

Backend	Python	3.10+
Framework web	Streamlit	1.28.0+
Base de datos	SQLite3	3.0+
Analisis de datos	Pandas	2.0.0+
Visualizacion	Plotly	5.18.0+
Calculos	NumPy	1.24.0+

10. Resultado esperado

- Base SQLite data_warehouse.db creada y poblada
- Validaciones de consistencia sin errores
- KPIs calculados y mostrados en consola
- Dashboard interactivo disponible en localhost

Documento generado a partir de README.md, documentacion/dashboard_documentacion.md y documentacion/bitacora_carga.txt.